

VPN Provisioning Portal

Workflows, system-to-system data flows and decision trees — Catalyst 8000v IPsec VTI + eBGP lab · Nautobot · Cisco Network-as-Code (Terraform) · Robot Framework

Generated from the codebase in github.com/dcantor/cat8000v-ipsec. Every diagram describes what the code actually does (file and function names are given), not an idealised design.

Contents

1. Systems and where the data lives
2. Architecture and data flows
3. The intent document
4. Workflow: Deploy / Dry run
5. Workflow: Add a spoke
6. Workflow: Add a headend
7. Workflow: Remove a spoke
8. Workflow: Resume, tests, inventory
9. Decision trees: intent validation
10. Decision trees: spoke allocation
11. Decision trees: the Nautobot seed
12. Decision trees: the renderer
13. Decision trees: apply, convergence and Golden Config
14. Decision trees: bootstrap and VM wiring
15. Decision trees: removal
16. REST API reference (data in / data out)
17. Test coverage map
18. File map

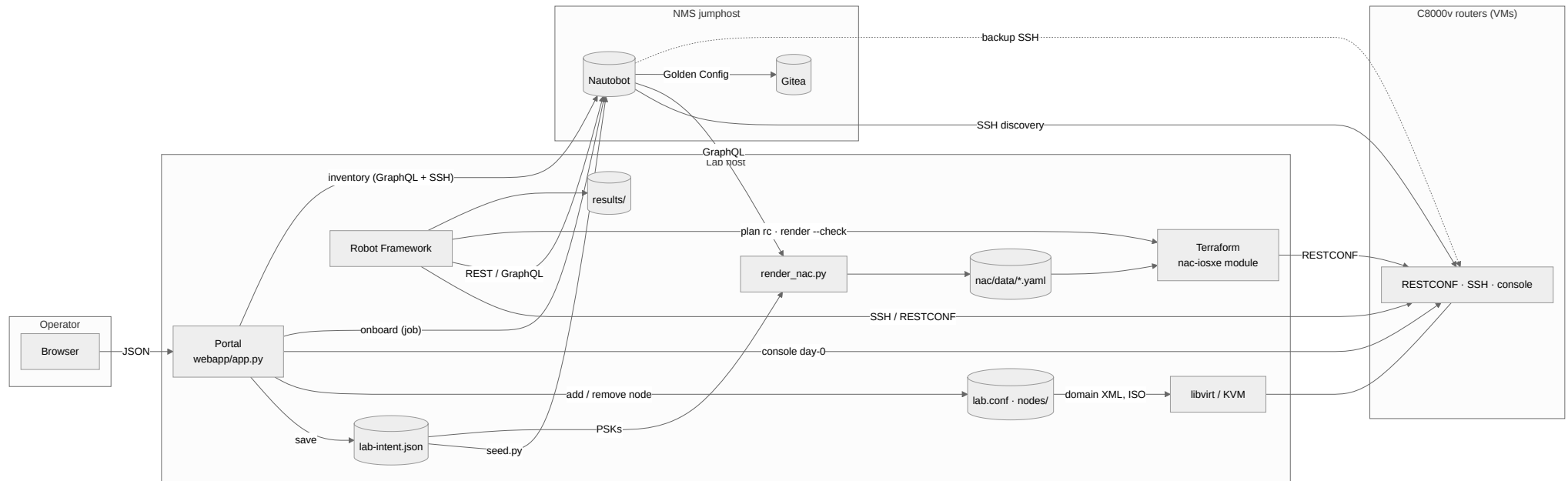
1. Systems and where the data lives

System	Runs where	Holds (system of record for)	Talks to
VPN Provisioning Portal FastAPI + single-page UI, <code>webapp/</code>	Lab host, <code>systemd --user lab-webapp</code> , port 8090	Run records (<code>webapp/runs/<id>.json</code> + the intent each run deployed); nothing else — it orchestrates	Browser (HTTP/JSON), every tool below via <code>./lab.sh</code> subprocesses, Nautobot (REST/GraphQL), headends (SSH, for the inventory)
lab-intent.json <code>nautobot/intent.py</code>	Lab root (git)	The VPN service intent: site/VPN metadata, regions, devices (hostname, mgmt IP, role, AS, router-id, LAN, region, site, site code, contact, per-spoke PSK), links, tunnels, crypto profile, capacity	Read by seed, renderer (PSKs), tests, portal; written by the portal and the spoke/hub/remove steps
lab.conf + libvirt <code>lab.sh</code>	Lab host, <code>qemu:///system</code>	VM facts: node names, node index (MACs, UDP ports), console ports, management IPs, physical wiring (<code>LINKS</code>), tunnel numbers, hub/spoke port counts; domain XML, overlay disks, day-0 ISOs under <code>nodes/</code>	libvirt (virsh), routers' serial consoles (raw TCP, <code>tools/console.py</code>)
Nautobot 3.2 (+ bgp-models, golden-config, device-onboarding, Gitea) NMS VM 10.0.0.10	Docker Compose on the NMS jumphost	Source of truth: locations (lab site → Region → Branch), devices, interfaces, cables, prefixes/IPs with roles, core VPN app objects (VPN, tunnels, endpoints, profile, Phase 1/2 policies), BGP AS/routing instances/peerings, config contexts, custom fields, saved GraphQL query, Golden Config settings/compliance; Gitea holds backups, intended configs, Jinja templates	Routers (SSH for onboarding/backups), seed/renderer/portal (REST + GraphQL), Gitea (git)

Network-as-Code nac/ : module netascode/nac- iosxe , provider CiscoDevNet/iosxe 0.15	Lab host, Terraform CLI	global.nac.yaml (hand-written baseline), generated devices.nac.yaml and device_groups.nac.yaml (PSK variables), Terraform state (terraform.tfstate)	Routers over RESTCONF (HTTPS 443, YANG native models); a cisco- ia:save-config RPC after apply
Catalyst 8000v routers	libvirt VMs on the lab host	Running/startup configuration; live IKEv2/IPsec/BGP state	Each other (UDP-tunnelled p2p links, IPsec VTIs, eBGP), NMS via OOB network 10.2.0.0/24 (Mgmt-vrf)
Robot Framework tests/	Lab host venv	Results per run under results/<date>_<time>/ (report, log, pre/post config backups, diff)	Routers (SSH via netmiko, RESTCONF), Nautobot (REST/GraphQL), Terraform (plan exit code), renderer (-- check), Gitea (raw backups)

What is deliberately not in Nautobot: pre-shared keys (one per spoke; they live only in lab-intent.json and are rendered as NAC variables into nac/data/device_groups.nac.yaml) and device credentials (Nautobot secrets group lab-devices resolves them from environment variables on the NMS). **What is deliberately not in the intent:** anything discovered from the routers (serials, model, platform) — that comes from Nautobot onboarding.

2. Architecture and data flows



Edge	Protocol	Data passed
Browser → Portal	HTTP JSON	Intent document; SpokeSpec / HubSpec / removal name; run options (golden, test, delete_disk); polling offsets
Portal → lab.conf / nodes/	file edits (regex on bash arrays)	New [name]= entries in ROLE/MGMT_IP/BGP_AS/LAN/CONSOLE_PORT/NODE_IDX, LINKS "hub:port spoke:port prefix", TUNNELS "id hub spoke prefix", ROUTERS/ALL_NODES; day-0 iosxe_config.txt from nodes/_template/ (hostname, mgmt IP, gateway substituted)
lab.sh → libvirt	virsh (define/start/undefine)	Domain XML: deterministic UUID (sha1 of the management IP → stable C8000v serial), Gi1 on the OOB bridge, GiN as UDP-socket NICs (pairs anchored on the headend: local 22000+idx·100+port, far 32000+idx·100+port), serial console TCP port, CVAC ISO
lab.sh → router console	raw TCP (tools/console.py)	Day-0 config (AAA, Mgmt-vrf/Gi1, SSH, NETCONF/RESTCONF, license boot level), RSA key generation, show version (license check) and a reload when needed
Onboarding → routers	SSH (netmiko/NAPALM inside Nautobot)	Hostname, model, serial, platform, interfaces; then onboard.py refreshes serials itself over SSH (the job does not update existing devices)
seed.py → Nautobot	REST (pynautobot) + GraphQL reads + two ORM writes via nbshell	Location types/locations, custom fields, roles, devices (matched by primary IP), interfaces (+MAC, enabled), IPs/prefixes (+roles, tags), cables, VPN policies/profile/VPN/tunnels/endpoints, AS/routing instances/peerings, config context, saved GraphQL query
Nautobot → renderer	GraphQL (saved query)	Per device: name, role, primary IP, config context, interfaces (type, enabled, IPs with parent prefix role/tags, connected interface, VPN endpoint → profile/policies, tunnel → far endpoint's source address), BGP routing instances with endpoints and peers
Renderer → NAC data	YAML files	devices.nac.yaml: per device system, crypto (proposal/policy/keyring peers/profile/transform set/IPsec profile), ethernet, loopbacks, native tunnels, BGP; CLI templates for ip tcp adjust-mss. device_groups.nac.yaml: group IPSEC_VPN with psk_<spoke> variables
Terraform → routers	RESTCONF (HTTPS, YANG Cisco-IOS-XE-native)	One resource per config object; PATCH on create/update, DELETE on destroy; -parallelism=1 to avoid datastore lock races; cisco-ia:save-config RPC per router after apply
Golden Config → routers / Gitea	SSH (nornir-netmiko), git	Backup: running-config → config-backups/<device>.cfg; Intended: Jinja2 c8000v-ipsec.j2 rendered from GraphQL (golden-config-lab query) → intended-configs/; Compliance: per-feature diff rows (17 features × N devices)
Robot → everything	SSH / RESTCONF / REST / GraphQL / subprocess	Assertions on live state vs the intent (lab_vars.py), on Nautobot objects, on Terraform drift (exit code), on renderer round-trip, on Golden Config compliance
Portal inventory → headends	SSH (netmiko)	show crypto ikev2 sa, show ip interface brief, show interfaces TunnelN, show bgp ipv4 unicast summary, show crypto ipsec sa → per-tunnel health, SA age, BGP state/prefixes, ESP counters, rates

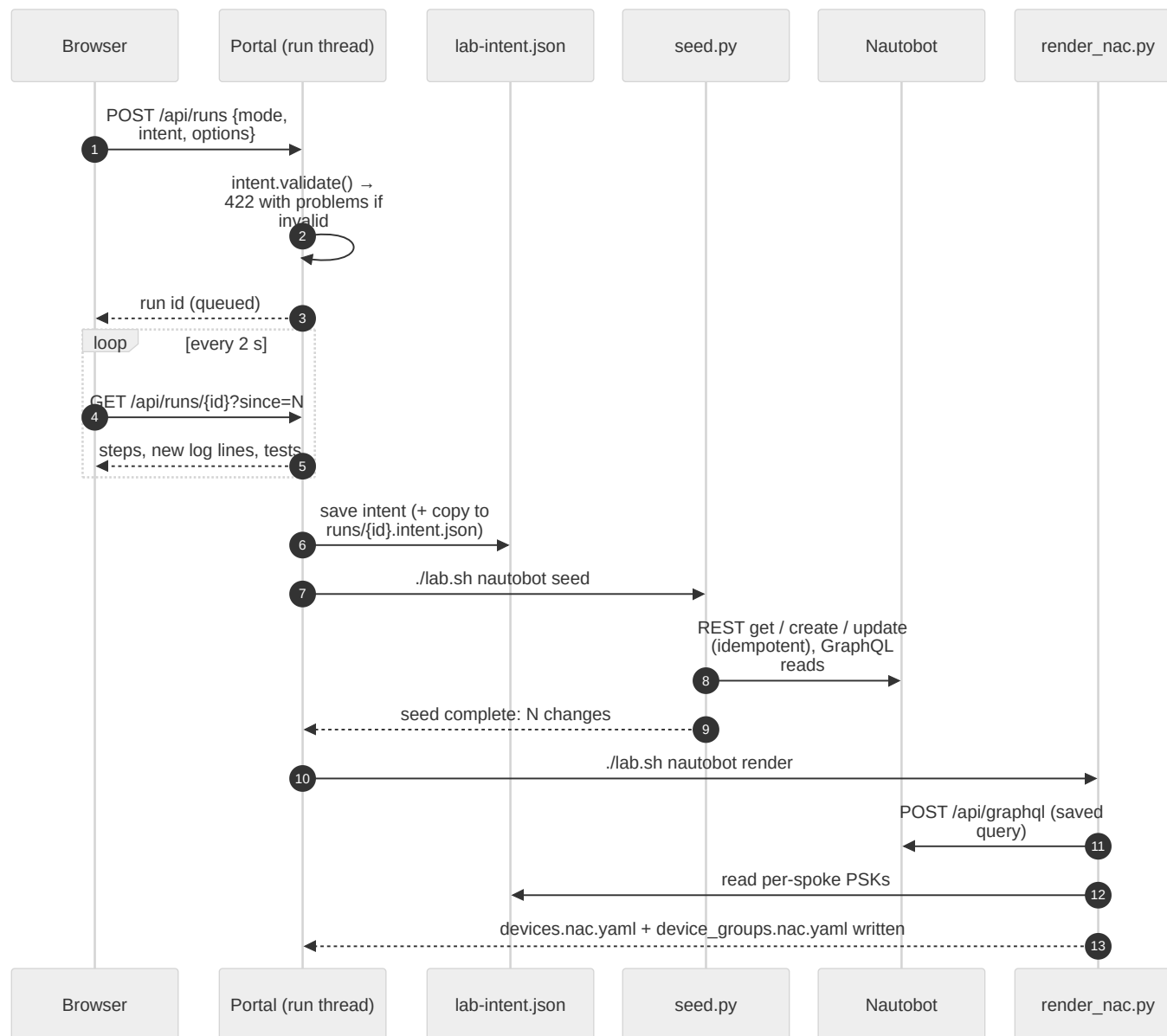
3. The intent document

`lab-intent.json` is the single input to every workflow. It is created once from `lab.conf` (`./lab.sh intent init`) and afterwards only written by the portal (deploy) or by the spoke/hub/remove steps. `intent.load()` upgrades older documents in place (tunnels gain a `hub`, spokes gain a `psk`, devices gain `region/site`).

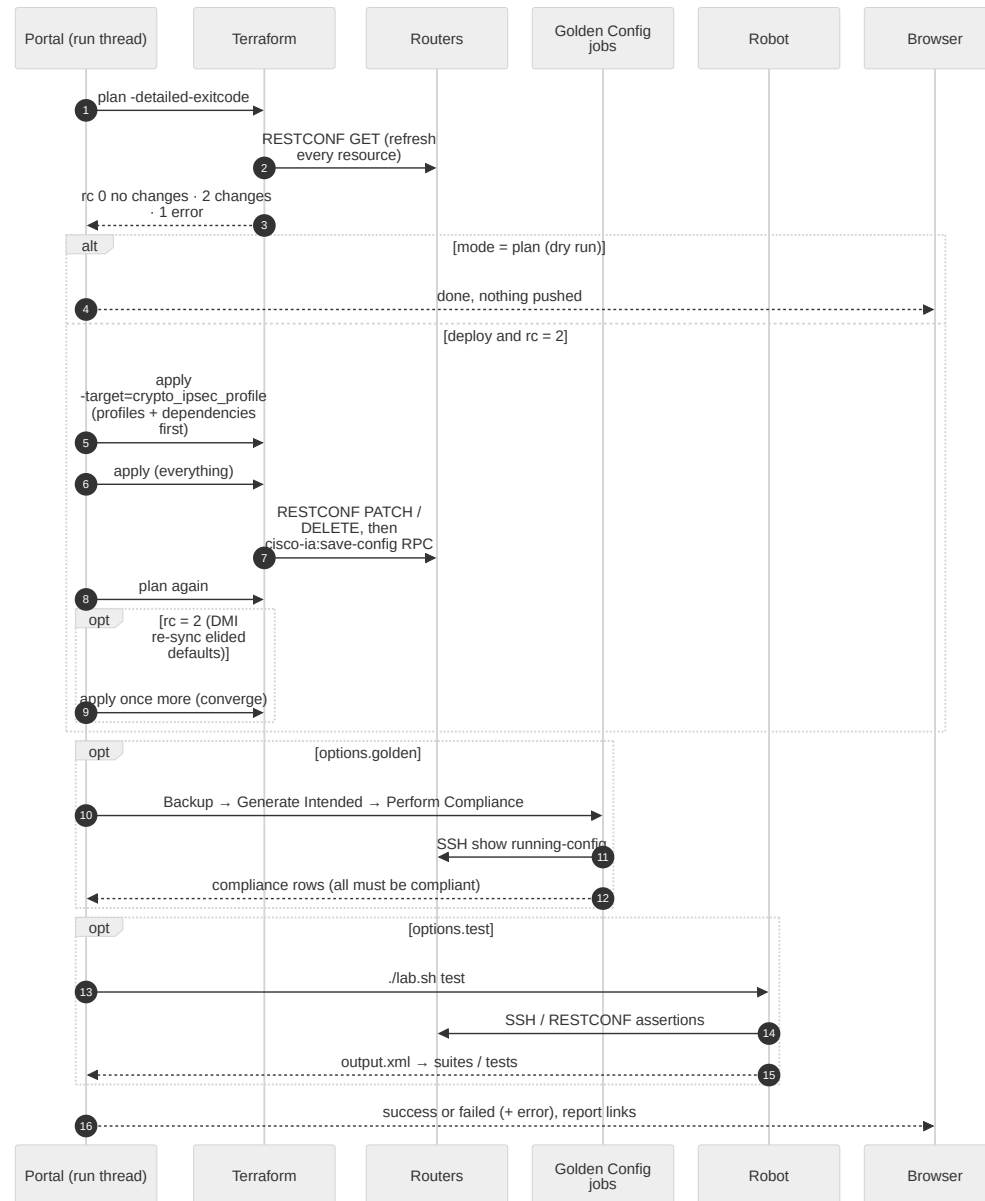
Key	Content	Consumed by
<code>site</code>	Lab-level location name (read-only), description, <code>site_code</code> , contact	seed (location + custom fields), tests (<code>NAUTOBOT_LOCATION</code>), inventory scope, Golden Config dynamic group
<code>vpn</code>	VPN name, description, <code>change_ticket</code> , owner	seed (VPN object + <code>extra_attributes</code>), run records, inventory/CSV
<code>profile</code>	name, <code>ike</code> {encryption, integrity, <code>dh_group</code> , lifetime}, <code>ipsec</code> {encryption, integrity, lifetime}, <code>dpd</code> {enabled, interval, retries}, <code>ios</code> object names	seed (Phase 1/2 policies, profile, <code>extra_options.ios</code>) → renderer/Golden Config read them back from Nautobot, never from the intent
<code>regions</code>	Ordered list; distance between regions = index difference	seed (Region locations), wizard (nearest two headends), inventory grouping
<code>devices[]</code>	name, <code>mgmt_ip</code> , role hub spoke, <code>asn</code> , <code>router_id</code> , <code>lan</code> , comments, region, <code>site</code> , <code>site_code</code> , contact, <code>psk</code> (spokes)	seed (devices matched by <code>mgmt IP</code> → renamed if needed; loopbacks; AS; branch location), renderer (PSK variables), tests, portal tables
<code>links[]</code>	<code>a</code> (headend), <code>a_port</code> , <code>b</code> (spoke), <code>b_port</code> , prefix /30 — must equal the physical wiring in <code>lab.conf</code>	validation (wiring check), seed (cables, WAN prefixes/IPs, interface enabled/description)
<code>tunnels[]</code>	<code>id</code> (TunnelN both ends), hub, spoke, prefix /30 — one per (hub, spoke)	seed (TunnelN interfaces, VPN tunnel + endpoints, BGP peering), tests
<code>oob</code> , <code>domain_name</code>	Mgmt-vrf, gateway, ACL name, OOB prefix; domain	seed (config context) → renderer/Golden Config via <code>config_context</code>
<code>capacity</code>	<code>tunnels_per_headend</code> (50)	validation (per hub), seed (custom field on hubs), inventory bars, wizard

4. Workflow: Deploy / Dry run

Triggered by **Deploy configuration** (mode `deploy`) or **Dry run** (mode `plan`) on the Provision page, or `POST /api/runs`. Steps: `validate` → `save` → `nautobot` → `render` → `plan` → `apply` → `golden` → `test` (plan stops after `plan`).

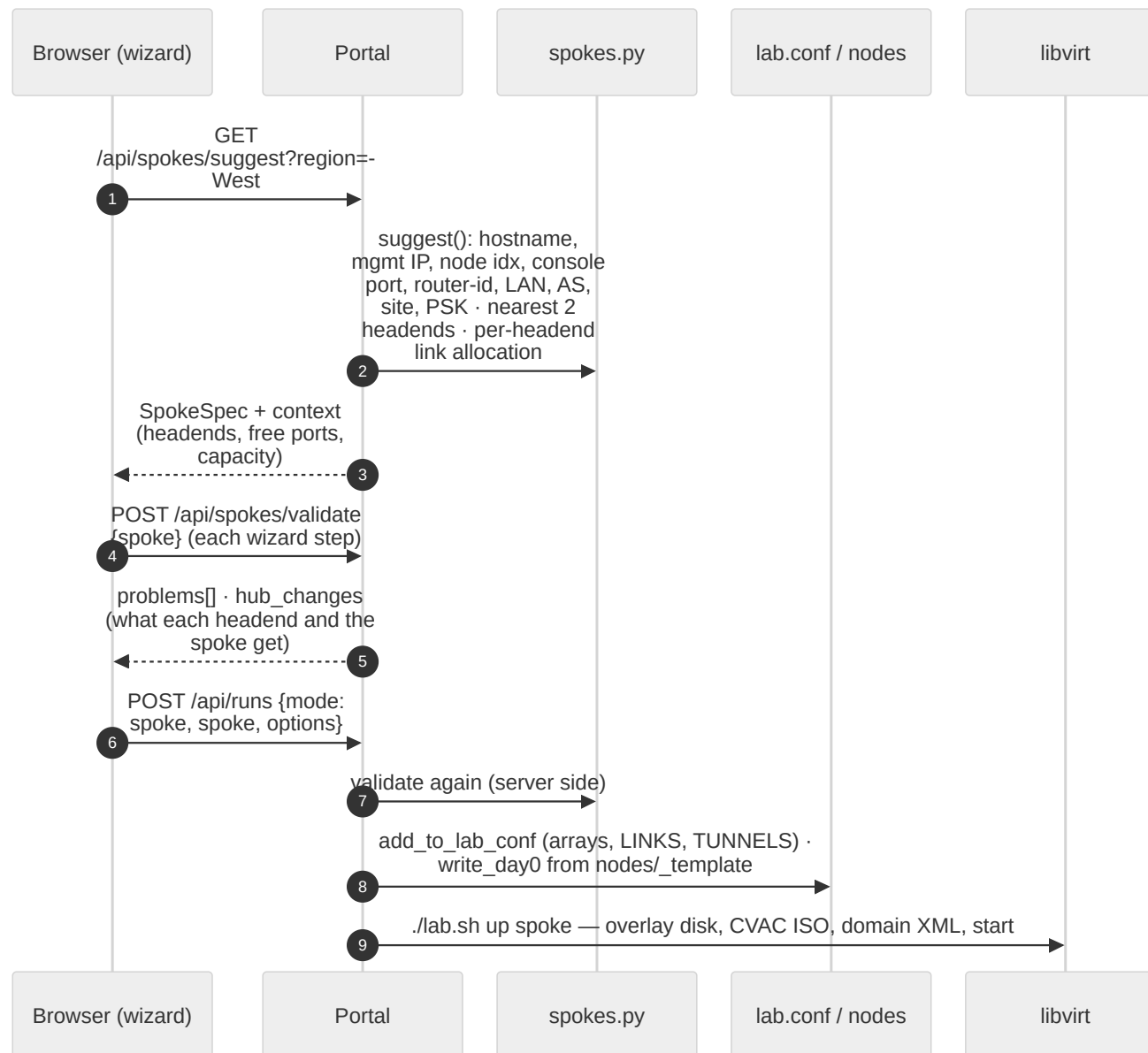


Phase 2 — plan, apply, verify

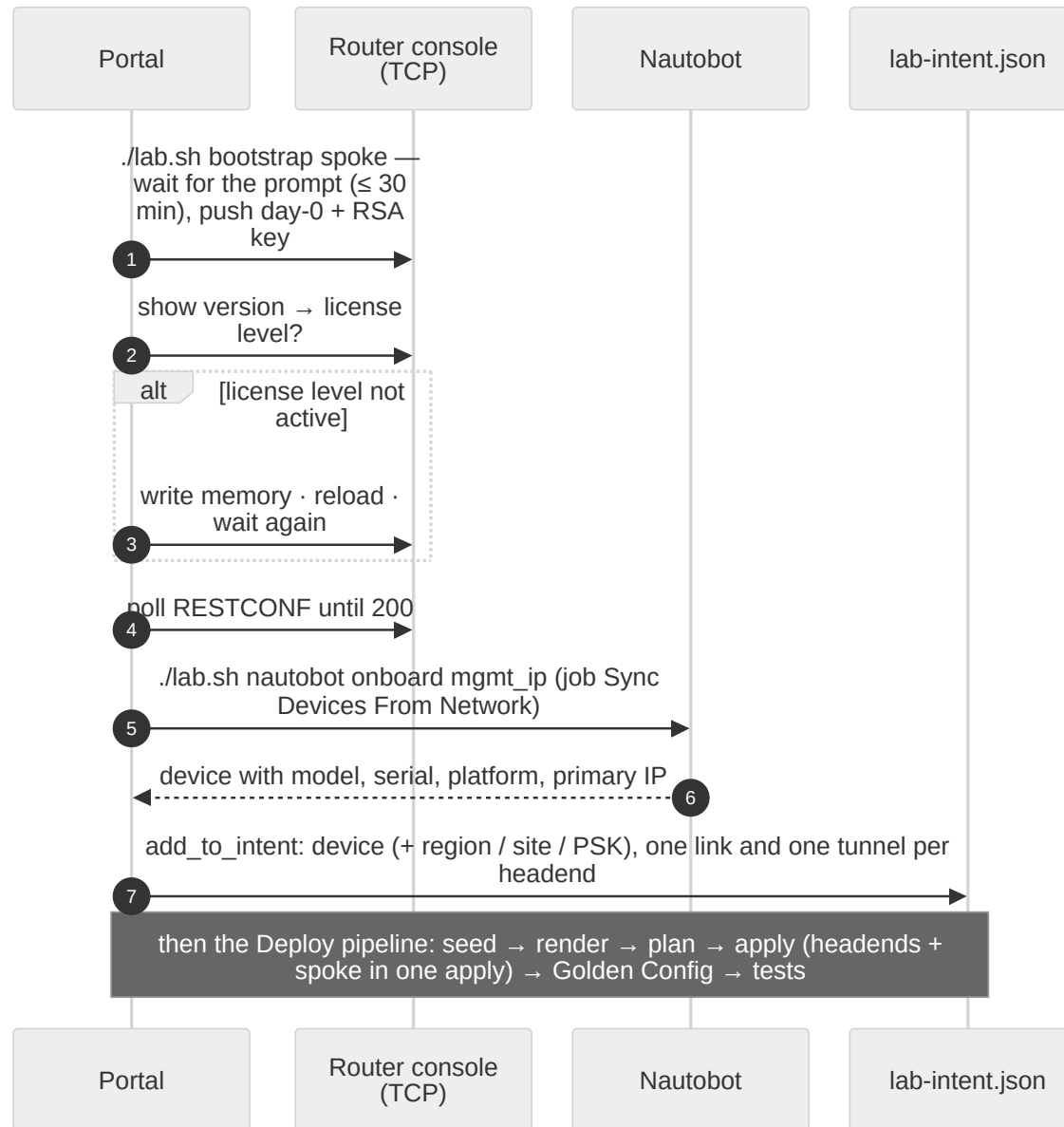


5. Workflow: Add a spoke

The wizard (3 steps) ends in `POST /api/runs {mode: "spoke", spoke: SpokeSpec}`. Steps: `spoke_validate` → `spoke_labconf` → `spoke_vm` → `spoke_bootstrap` → `spoke_onboard` → `spoke_intent` → `nautobot` → `render` → `plan` → `apply` → `golden` → `test`.



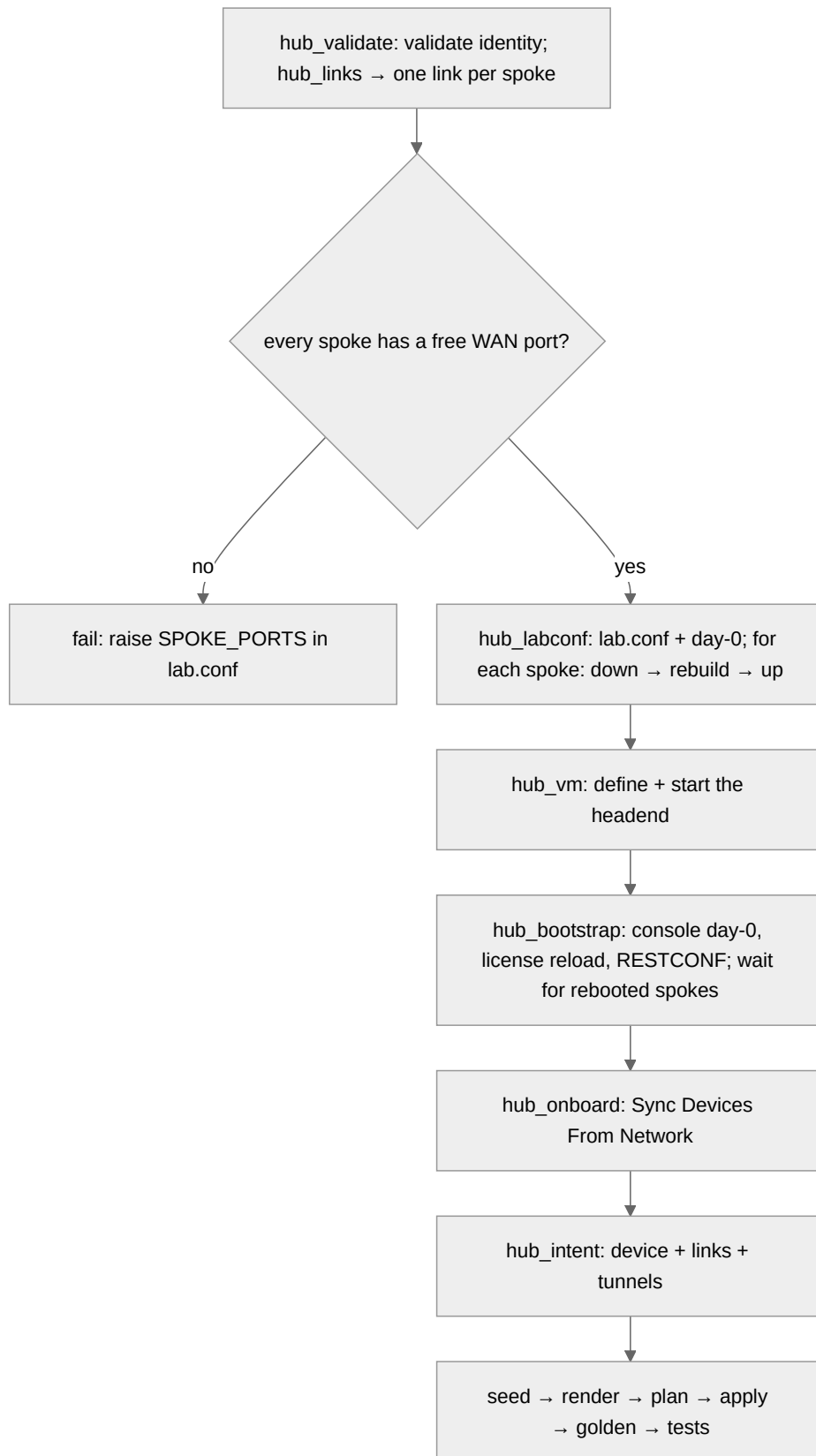
Phase 2 — bring the router up and into the model



Every step is idempotent so a failed run can be **resumed**: `spoke_labconf` skips if the name is already in `lab.conf`, `lab.sh up` reports "already running", `spoke_intent` skips if the device is already in the intent.

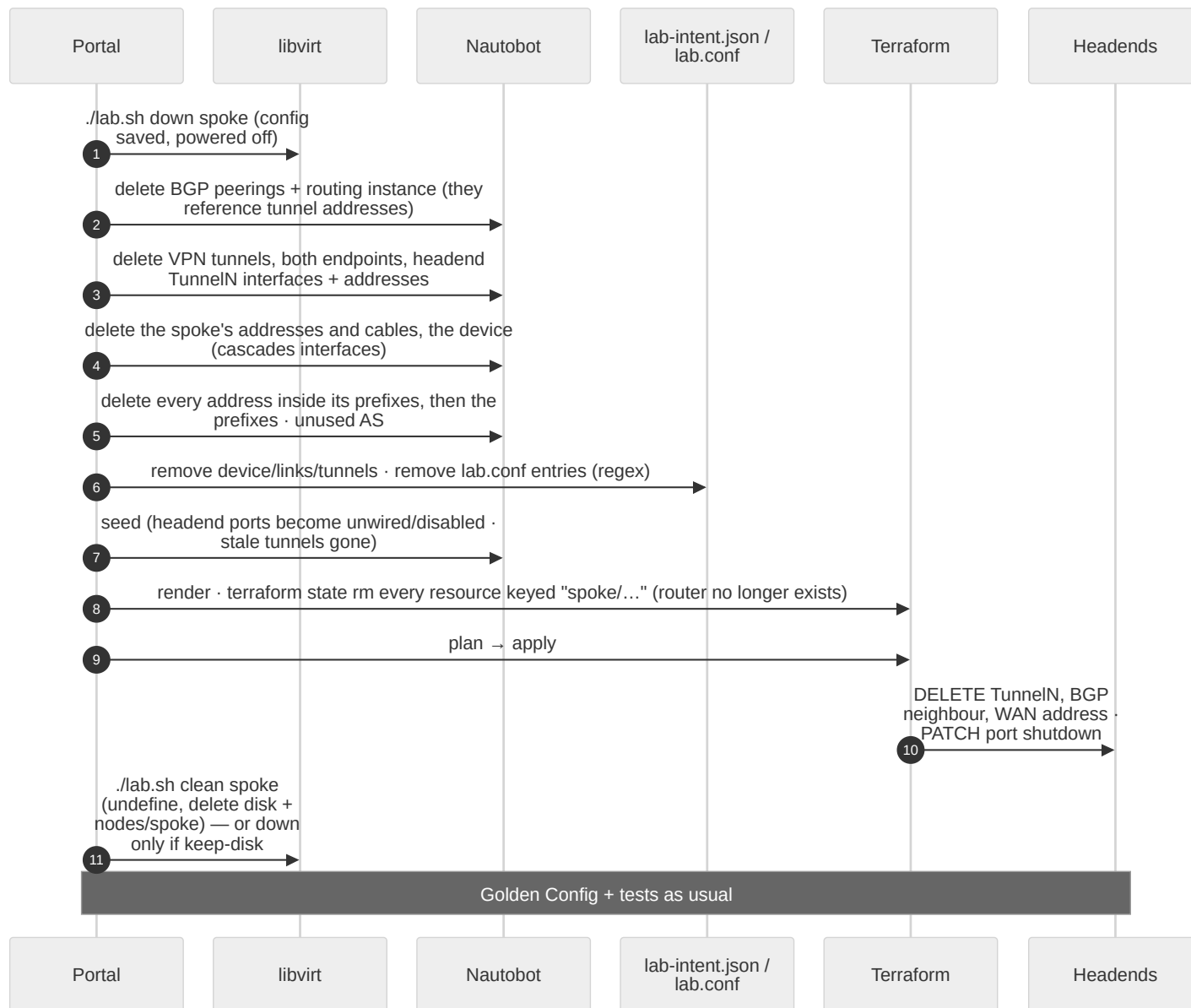
6. Workflow: Add a headend

`POST /api/runs {mode: "hub", hub: HubSpec}` (from `GET /api/hubs/suggest`). Same shape as a spoke, with two differences: the new headend gets a link + tunnel to **every existing spoke** (`spokes.hub_links()`, each spoke's next free WAN port), and because a link is anchored on the headend's UDP ports, each existing spoke's libvirt definition changes — the run re-defines and reboots each spoke (`down / rebuild / up`) and waits for them after the headend's bootstrap.



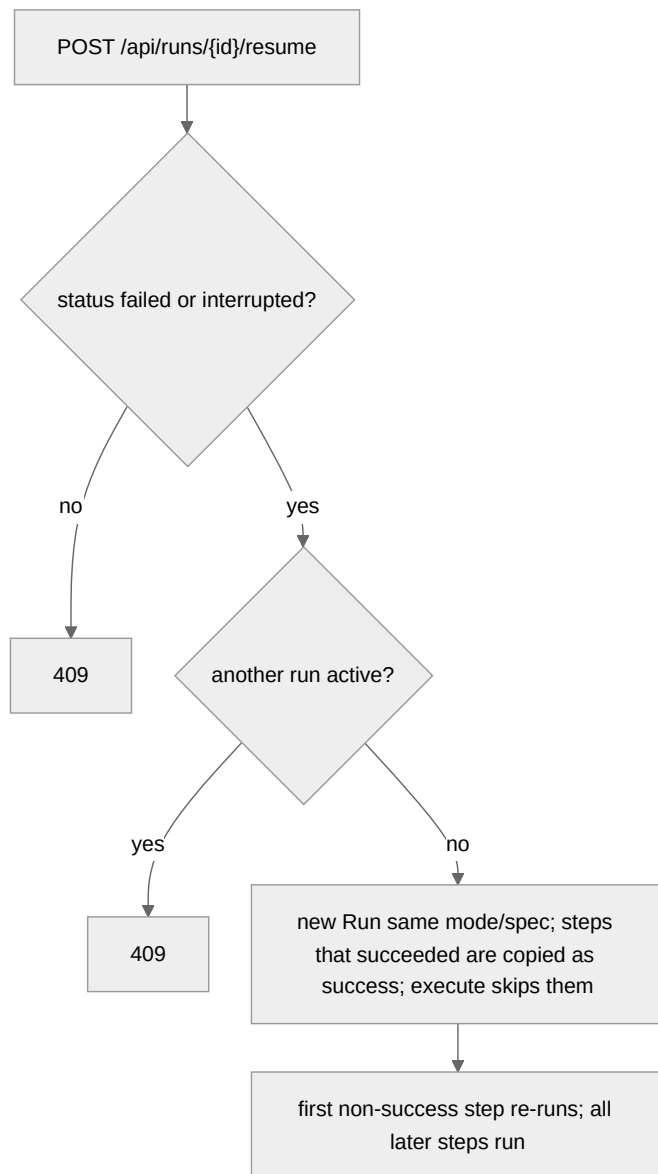
7. Workflow: Remove a spoke

Remove... on a spoke row → GET /api/spokes/{name}/removal (plan dialog) → POST /api/runs {mode: "remove", spoke: {name}}. Steps: rm_validate → rm_down → rm_nautobot → rm_intent → nautobot → render → rm_state → plan → apply → rm_vm → golden → test.



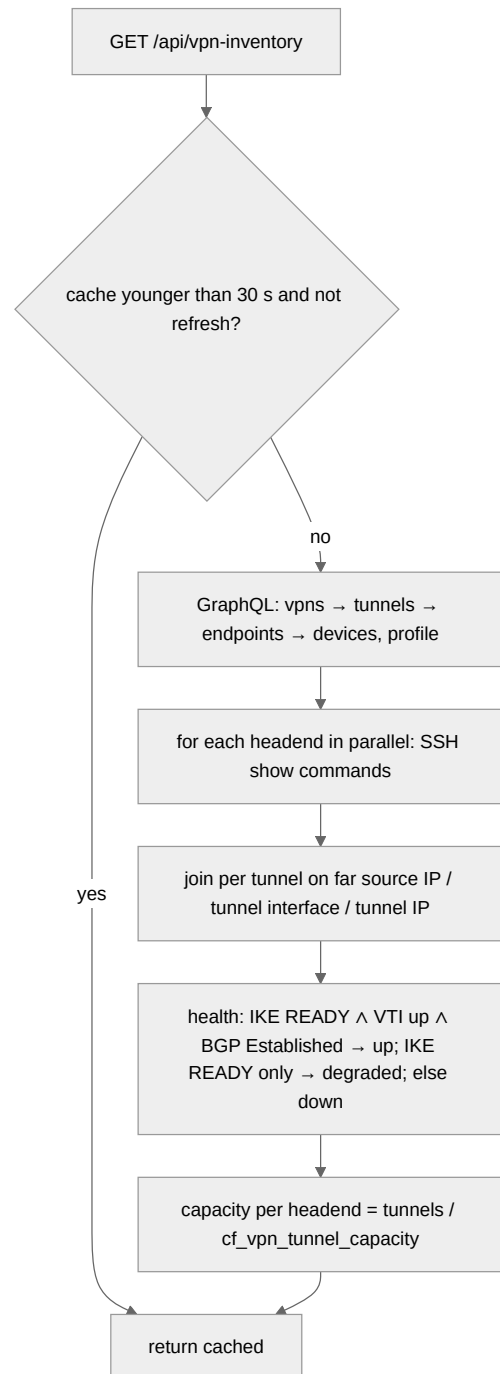
8. Workflow: Resume, tests, inventory

Resume



On start-up the portal marks runs found "running" as **interrupted** (running step → failed, pending → skipped) so they are resumable. `webapp/restart.sh` refuses to restart while a run is active.

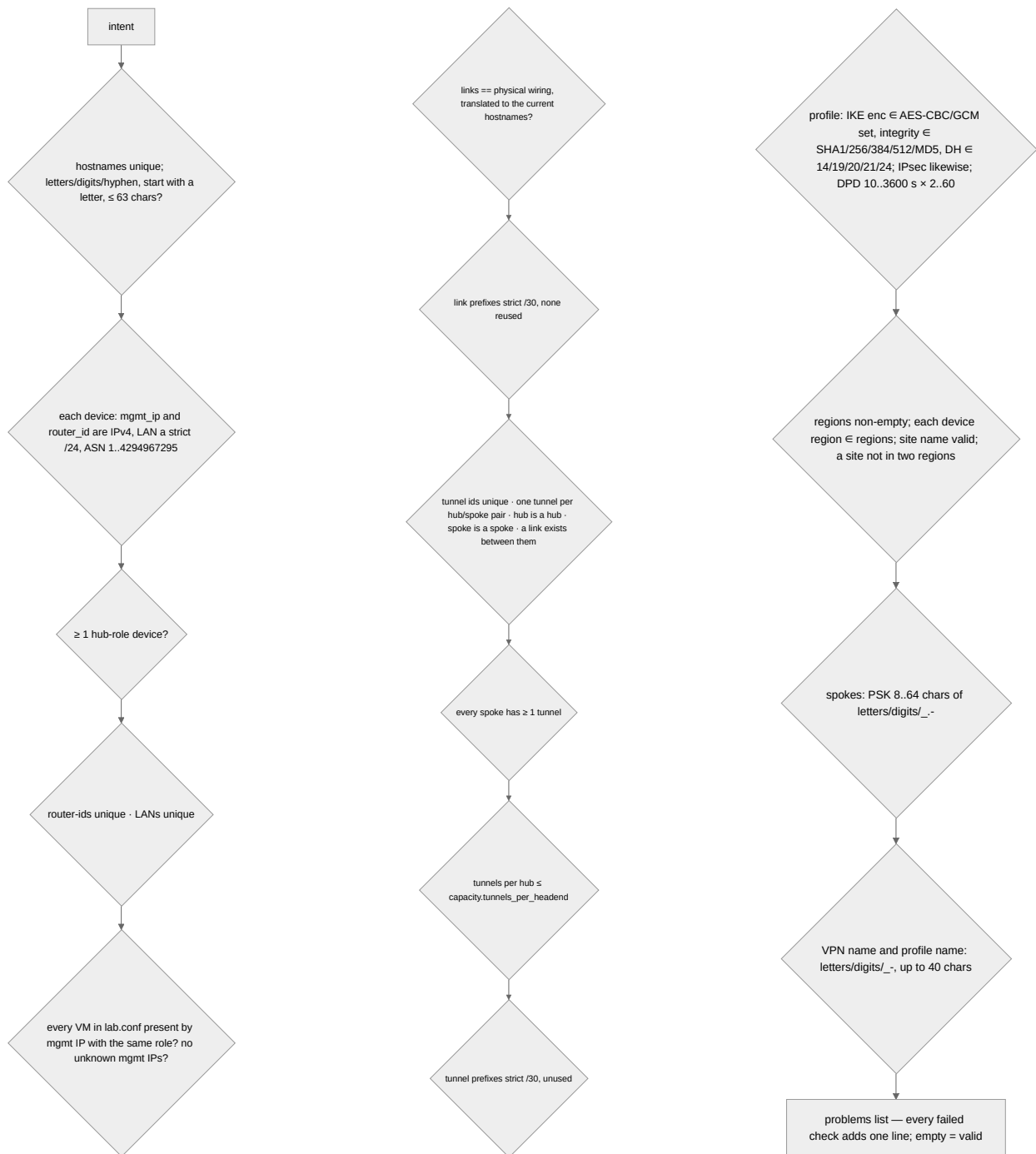
Inventory refresh



Tests (mode test , or the last step of every run)

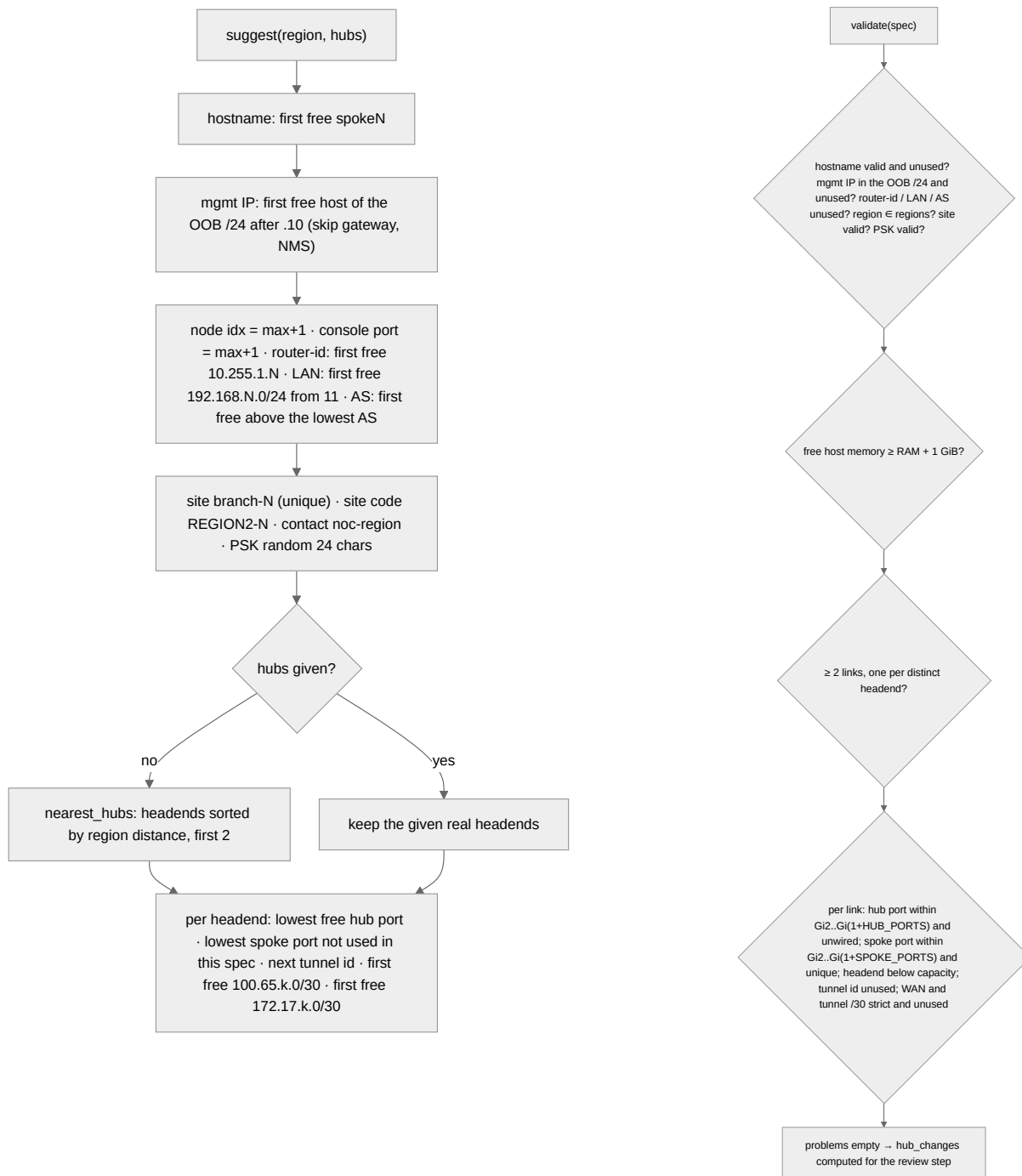
`tests/run.sh` creates `results/<date>_<time>/`, captures every router's running/startup config (`configs/pre-run`), runs the six suites, captures again (`configs/post-run`) and diffs; `results/latest` points at the newest run. The portal parses `output.xml` into the per-suite report shown in the UI and serves `report.html` / `log.html` / `backups` under `/results/`.

9. Decision trees: intent validation (`intent.validate`)

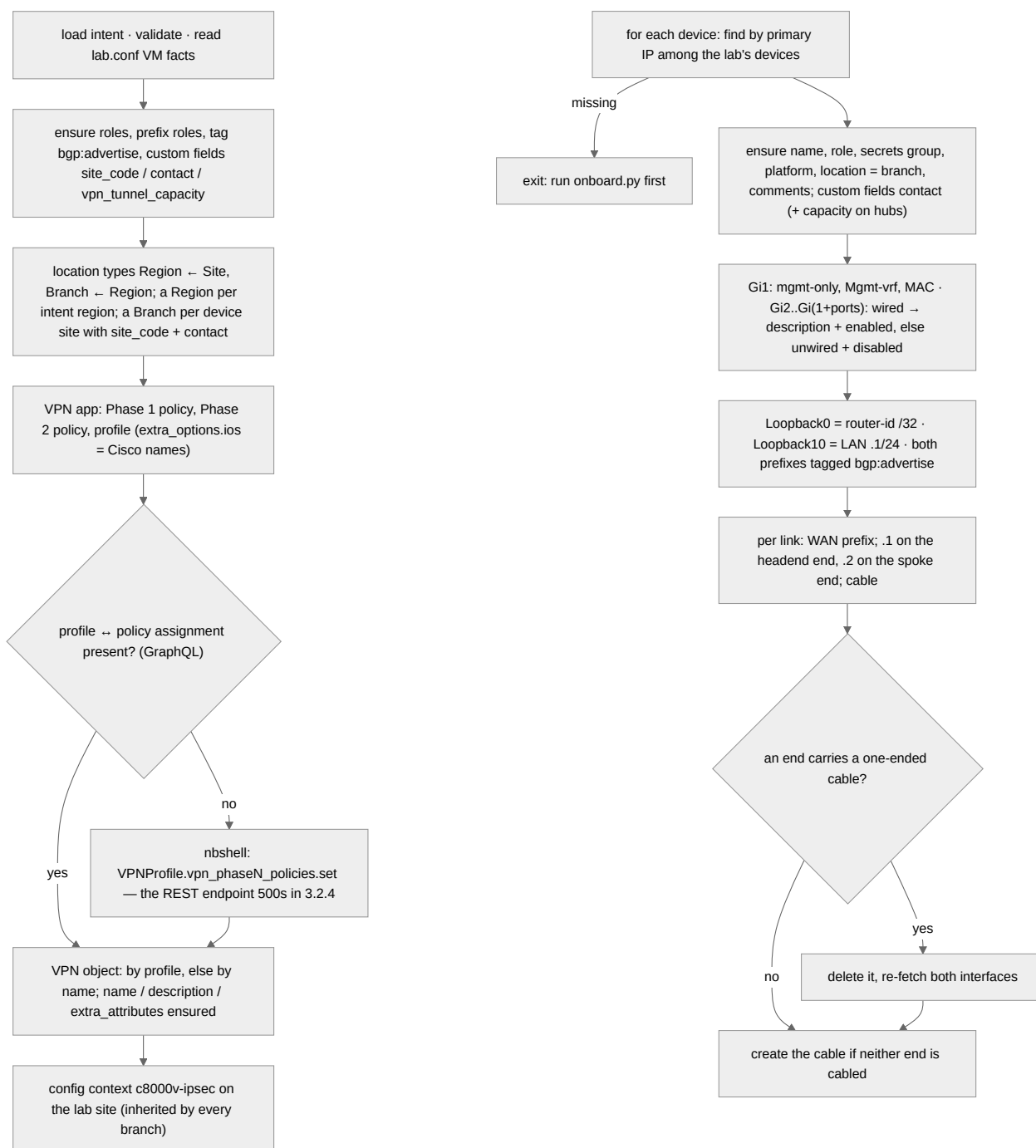


All checks run; the caller gets the full list. The pipeline refuses to start (HTTP 422) or fails its first step on any problem.

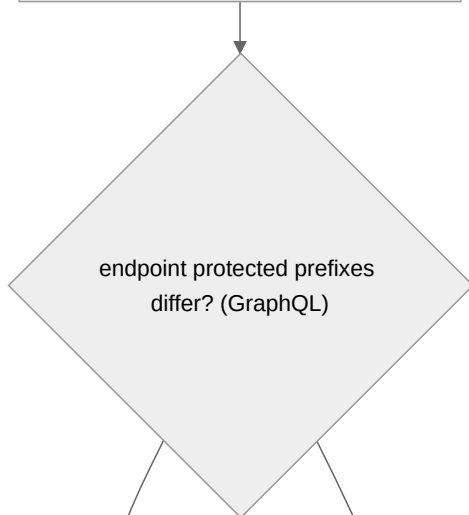
10. Decision trees: spoke allocation (`spokes.suggest` / `validate`)



11. Decision trees: the Nautobot seed (`nautobot/seed.py`)



per tunnel, both ends: delete other TunnelX with the same far-end description; TunnelN (type tunnel) + /30 IP; endpoint (role, source iface + IP, tunnel iface, protected prefixes)



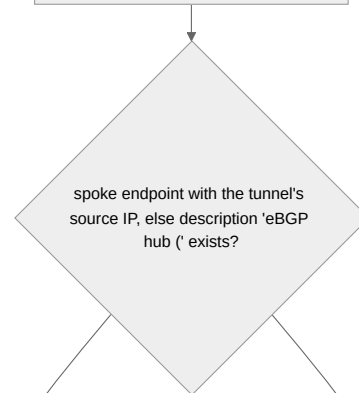
yes
direct PATCH (M2M absent from REST reads)

no

VPN tunnel: by vpn + tunnel_id, else by name; A = headend, Z = spoke; encapsulation IPsec-Tunnel

delete this VPN's tunnels that are not in the intent

BGP: AS per device; routing instance (router-id = Loopback0); ipv4_unicast AF



yes

re-point source IPs / AS / descriptions

no

create peering + both endpoints + endpoint AFs

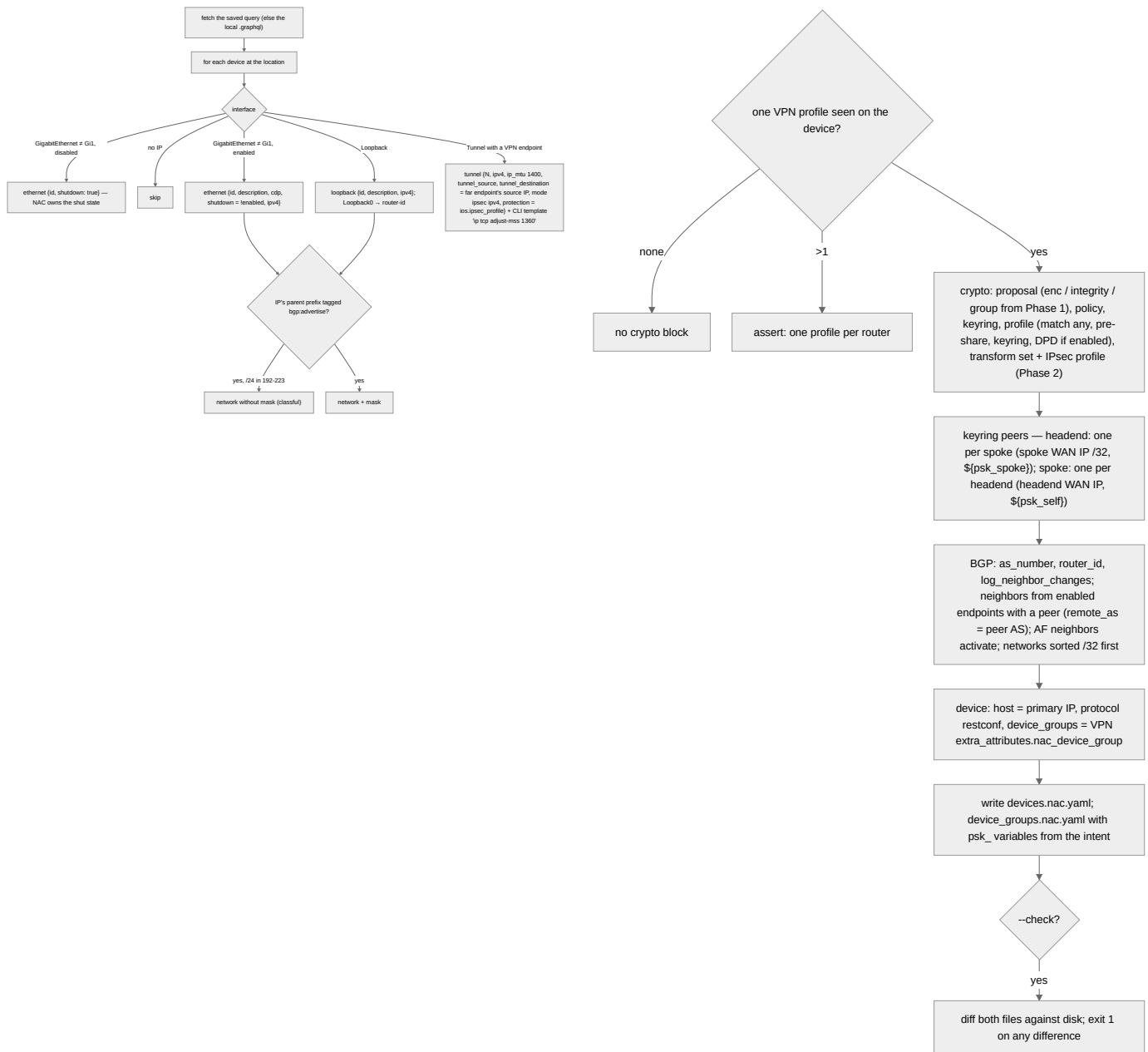
delete IPsec-VTI AS objects no device uses

saved GraphQL query: location + device names substituted; PATCH if the text differs

print N changes (0 on a re-run)

Comparison rule (`ensure()`): related objects compare by id, choice fields by value, MACs case-insensitively, JSON fields structurally (nested pynautobot Records serialised). Only fields that differ are PATCHed — a re-run reports **0 changes**.

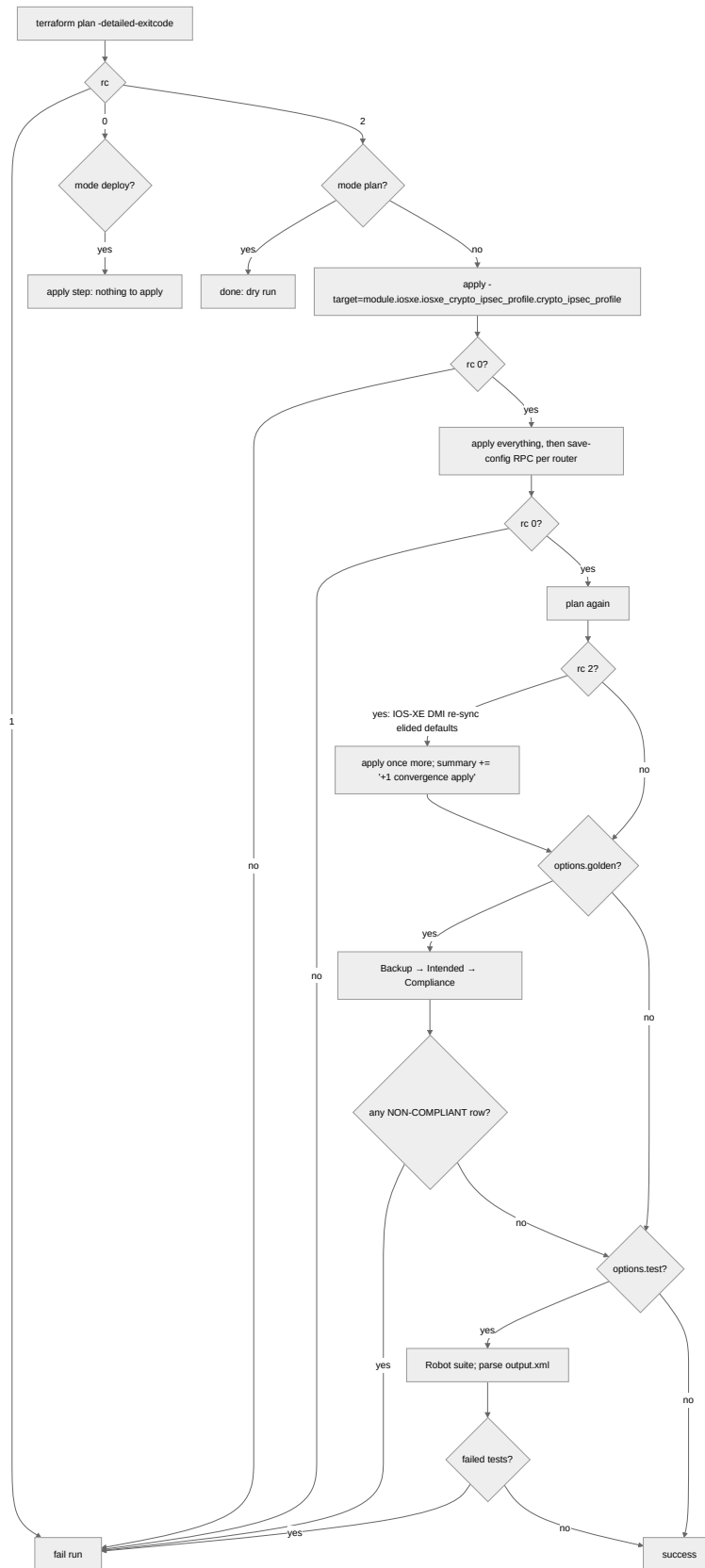
12. Decision trees: the renderer (`nautobot/render_nac.py`)



GraphQL renders single-choice fields as enum names (`IPSEC_TUNNEL` , `IKEV2`) but JSON-array choice fields raw (`AES-256-CBC`); `norm()` upper-cases and replaces "-" so both forms map to the same Cisco keywords.

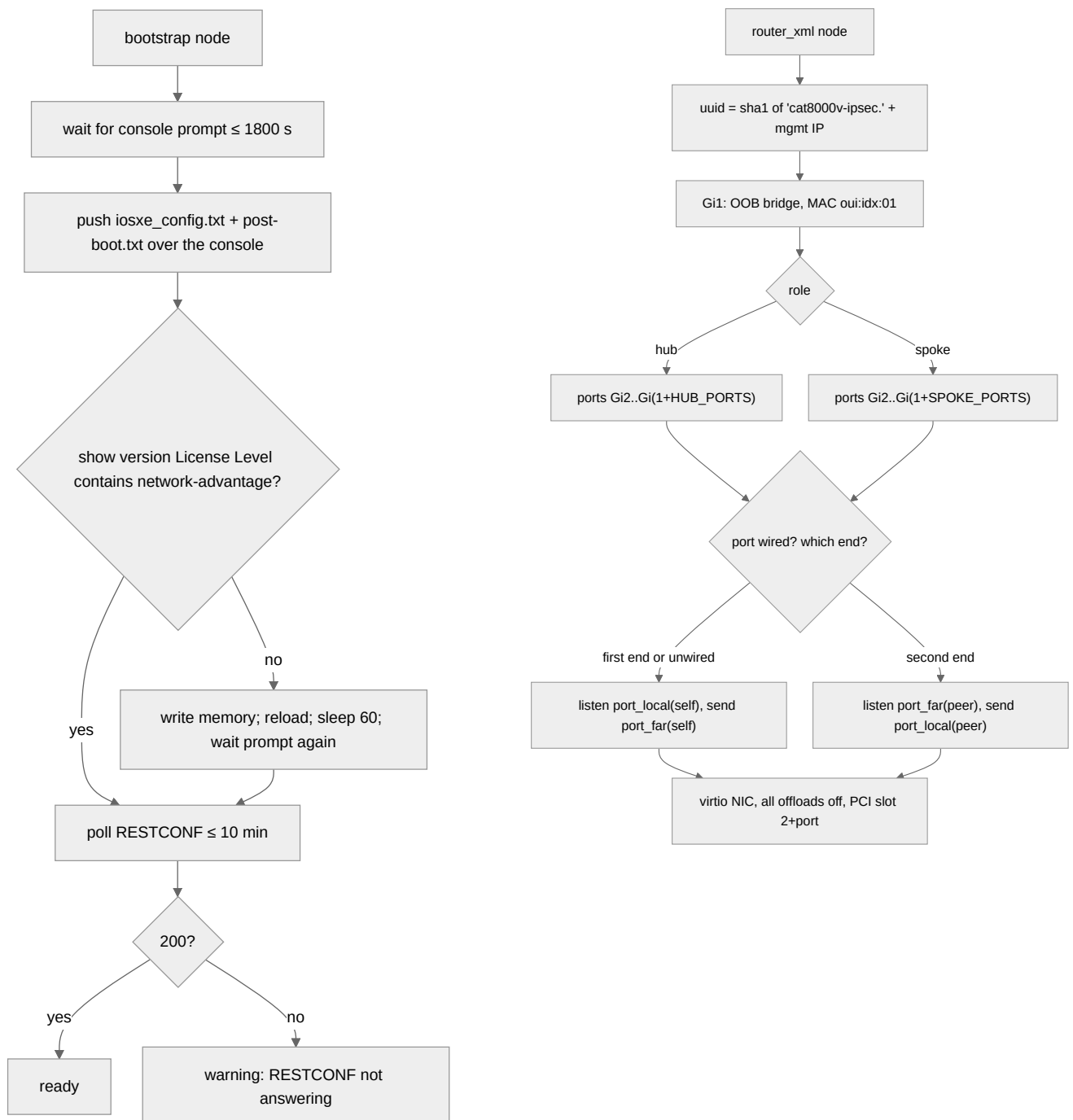
13. Decision trees: apply, convergence and Golden Config (app.py

do_plan/do_apply/do_golden)



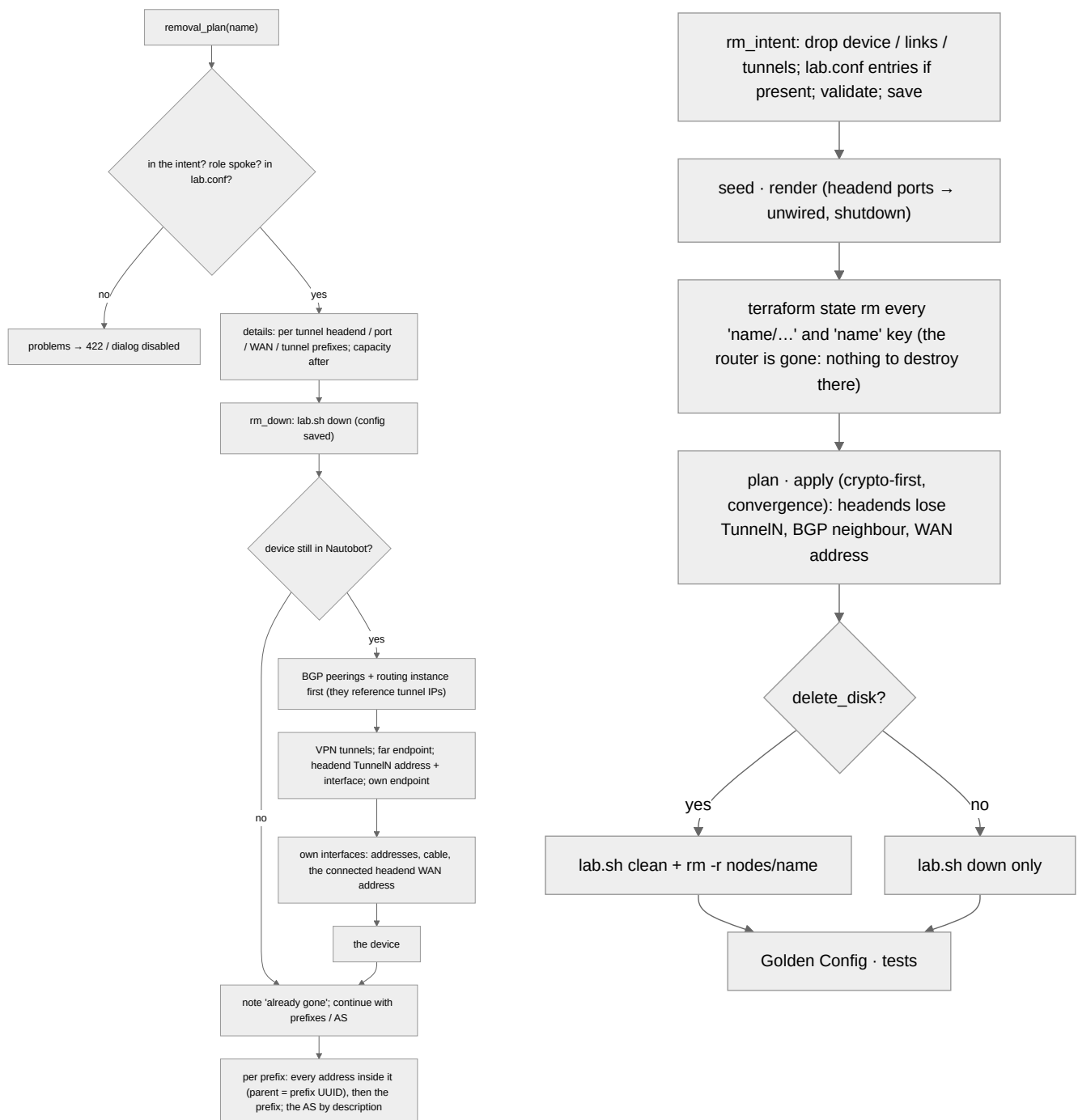
Why the crypto-profiles-first apply: the NAC module has no dependency from `iosxe_interface_tunnel` to `iosxe_crypto_ipsec_profile`, and IOS-XE refuses a tunnel whose `tunnel protection ipsec profile` does not exist yet — on a brand-new router the first full apply could push the VTI before its profile. **Why the convergence apply:** after an interface deletion or a reload IOS-XE re-syncs its YANG datastore and elides default values (transform-set key size, SSH defaults); one re-assert fixes it.

14. Decision trees: bootstrap and VM wiring (lab.sh)



Because a link's UDP pair is anchored on its first end (always the headend), adding a spoke never changes a headend's domain; adding a headend changes each spoke's definition (its new port's target), hence the spoke reboots in the hub workflow.

15. Decision trees: removal (`spokes.removal_plan` / `remove_from_nautobot`)



16. REST API reference (data in / data out)

Interactive documentation with schemas and examples: `/docs` (Swagger UI) and `/redoc`; OpenAPI JSON at `/openapi.json`.

Endpoint	Input	Output
GET <code>/api/intent</code>	—	intent, wiring (physical links), nodes (mgmt IP → VM name/idx/role/console), Nautobot public URL
GET <code>/api/inventory</code>	—	Nautobot devices at the lab location (name, mgmt IP, serial, model, role, status, URL), VPN URL
POST <code>/api/validate</code>	<code>{intent}</code>	<code>{problems: []}</code>
GET <code>/api/spokes/suggest?hubs=@ion=</code>	optional headends, region	SpokeSpec (all fields) + context (headends with region/distance/usage/free ports, regions, nearest, capacity)
GET <code>/api/hubs/suggest</code>	—	HubSpec + context (spokes to connect)
POST <code>/api/spokes/validate</code>	<code>{spoke: SpokeSpec}</code>	<code>{problems, hub_changes: {links: [per headend: interface, wan_ip, tunnel, tunnel_ip, tunnel_destination, bgp_neighbor, spoke{...}, tunnels_after, capacity], loopbacks}}</code>
GET <code>/api/spokes/{name}/removal</code>	name	<code>{problems, details: {mgmt_ip, asn, lan, router_id, links[...]}}</code>
POST <code>/api/runs</code>	<code>{mode, intent spoke hub, options{golden,test,delete_disk}}</code>	Run (id, mode, status, steps[name,title,status,started,finished,summary]); 409 if a run is active; 422 with problems
GET <code>/api/runs</code>	—	last 30 runs without logs (memory + <code>webapp/runs/*.json</code>)
GET <code>/api/runs/{id}?since=N</code>	log offset	Run + <code>log[{t,line}]</code> from N, <code>log_offset</code> , tests (suites → tests with status/message/elapsed), <code>results_dir</code> , devices, <code>resume_of</code>
POST <code>/api/runs/{id}/resume</code>	—	new Run with carried-over successes; 404/409
GET <code>/api/vpn-inventory?refresh=&live=</code>	flags	tunnels (model + live), headends (capacity, used, free, utilisation, live status), devices (role, region, site, mgmt IP, AS, LAN, serial), vpns, summary, generated
GET <code>/api/vpn-inventory.csv</code>	—	CSV, one row per tunnel (35 columns)
<code>/results/...</code>	path	static Robot reports, logs, config backups

17. Test coverage map

Suite	Proves	Sources compared
01 Management	SSH/RESTCONF reachability, hostname, IOS-XE version, Mgmt-vrf default route, license level, AAA/SSH/VTY hardening, banner	routers vs intent + NAC baseline

02 Underlay	every hub ↔ spoke p2p link up with the modelled /30s, ping both ways, no spoke ↔ spoke WAN reachability, CDP neighbour per link, loopbacks	routers vs intent
03 IPsec VTI	tunnel config lines, up/up, IKEv2 SAs READY with the modelled suite, DPD, ESP counters advance (encryption proven), sessions per headend = its tunnels	routers vs intent
04 Routing	eBGP Established headend ↔ spoke, spokes peer only with their headends, every site learnt, spoke-to-spoke AS path via a headend, LAN reachability, traceroute first hop is a headend	routers vs intent
05 NAC compliance	<code>terraform plan</code> exit code 0 (no drift)	routers vs NAC data
06 Nautobot	devices/serials/locations, cables and /30s, VPN app objects and that each router's tunnel destination equals the far endpoint's source address, crypto policies vs routers, BGP model vs live sessions, <code>render --check</code> round-trip, Golden Config backups in Gitea and every compliance row compliant	Nautobot vs routers vs committed NAC data

18. File map

Path	Purpose
<code>lab.conf</code> , <code>lab.sh</code> , <code>tools/console.py</code> , <code>nodes/</code>	VM facts and libvirt controller (<code>up/down/bootstrap/rebuild/clean/rename/status/nac/nautobot/test/webapp</code>), console automation, day-0 template and per-node files
<code>lab-intent.json</code> , <code>nautobot/intent.py</code>	The intent document; load/upgrade/validate/save, wiring & VM facts, region distance, nearest headends, PSK generation
<code>nautobot/onboard.py</code> , <code>seed.py</code> , <code>render_nac.py</code> , <code>golden_config.py</code> , <code>rename_node.py</code> , <code>nac-c8000v-ipsec-</code> <code>model.graphql</code> , <code>golden-config-</code> <code>templates/c8000v-ipsec.j2</code>	Nautobot integration: discovery, intent → objects, objects → NAC data, Golden Config setting/template/jobs, rename tool, the saved query, the Jinja template
<code>nac/main.tf</code> , <code>versions.tf</code> , <code>data/*.nac.yaml</code>	Terraform root: module + provider pins, baseline, generated device model and PSK variables
<code>tests/</code>	Robot suites 01–06, <code>LabLib.py</code> keywords, <code>lab_vars.py</code> (derived from the intent), <code>run.sh</code> , config capture
<code>webapp/app.py</code> , <code>schemas.py</code> , <code>spokes.py</code> , <code>inventory.py</code> , <code>static/index.html</code> , <code>runs/</code> , <code>demo/</code>	Portal API + pipeline runner, OpenAPI models, spoke/headend allocation and removal, inventory collector, UI, run records, demo recorders